Naive Bayes: A Worked Example in Code

Contents

1. Overview 2
2. The data 2
3. Two phases 3
4. Training: the lookup table 4
5. Testing: look up and multiply 6
6. The zero-frequency problem 7
7. The fix: Laplace smoothing 8
8. Summary 11
9. Sources 11
10. Key terms 11

Where this fits: Pipeline map step 8 (Model). See the Course map.

0 Foundations	1 Frame	2 Get data	3 Understand	4 Clean	5 Features	6 Reduce
7 Split	**8 Model**	9 Evaluate	10 Tune	11 Deploy	12 Test	13 Monitor

- **Builds on:** Naive Bayes.

1. Overview

Key point: Training Naive Bayes means building a **lookup table** (G-1128) of probabilities by counting. Predicting means looking up one probability per feature and multiplying.

The intuition and the formula of Naive Bayes (a classifier that scores each class by multiplying one probability per feature) are applied here to a classic toy dataset, **Play Tennis**, in Python: first by hand with pandas, then with scikit-learn.

This Note covers:

- the data and the question (section 2);
- the two phases, training and testing (section 3);
- training: building the lookup table by counting (section 4);
- testing: looking up and multiplying (section 5);
- a practical problem, the **zero-frequency problem** (G-2149) (section 6);
- its standard fix, **Laplace smoothing** (G-1045), and scikit-learn's `CategoricalNB` (section 7).

2. The data

Key point: 14 days, four weather features, and whether tennis was played (9 yes, 5 no).

Each day is one **observation** (one record, a row of the data table). Each has four **features** (input variables, one column each):

- **outlook:** sunny, overcast, rain;
- **temperature:** hot, mild, cool;
- **humidity:** high, normal;
- **wind:** weak, strong.

The **target** (the output we predict) is **play** (yes or no). Tennis was played on 9 of the 14 days.

The question: on a day that is sunny, hot, high-humidity with weak wind, will tennis be played?

From the formula, each class gets a score: its **prior** (G-1565; how common the class is, here $P(\text{yes})$ is the share of days with tennis) multiplied by one **likelihood** (G-1086; how often a

feature value occurs within the class) per feature. The bar in $P(\text{sunny} \mid \text{yes})$ reads "given": the probability of sunny among the days with tennis (see the notation).

$$\begin{aligned} \text{score}(\text{yes}) &= P(\text{yes}) \\ &\times P(\text{sunny} \mid \text{yes}) \times P(\text{hot} \mid \text{yes}) \\ &\times P(\text{high} \mid \text{yes}) \times P(\text{weak} \mid \text{yes}) \end{aligned}$$

The score for "no" has the same form. We predict the class with the larger score, which is the **MAP rule** (G-1157; maximum a posteriori: pick the class with the largest posterior probability, the probability of the class after seeing the features, see the names of the four parts).

3. Two phases

> **Key point:** Training computes and stores every probability; testing only looks them up.

Like every machine learning algorithm, Naive Bayes works in two phases:

1. **Training:** go through the data once and compute every probability the formula could ever need: the class priors, and $P(\text{value} \mid \text{class})$ for every value of every feature. Store them in a **lookup table** (G-1128) (in Python, a dictionary).
2. **Testing:** for a new day, look up the relevant probabilities and multiply. Nothing is recomputed.

How many probabilities does the table hold? Outlook has 3 values and there are 2 classes, so 6; temperature 6; humidity 4; wind 4; plus 2 priors. 22 numbers in all.

Figure 1 shows the two phases side by side. Watch the dashed arrow: testing only reads the table that training built.

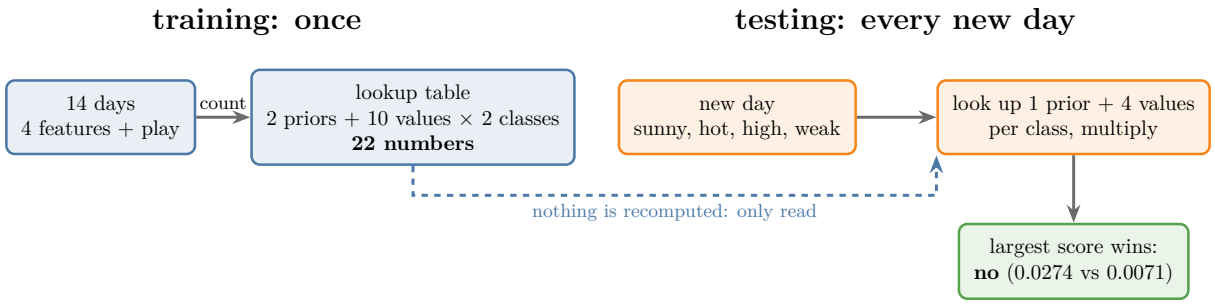


Figure 1: Training counts the 14 days once into a 22-number lookup table; testing reads 1 prior and 4 values per class for each new day and multiplies

4. Training: the lookup table

Key point: pd.crosstab counts each value per class; dividing by the class sizes gives P(value | class).

Python: Building the **lookup table** (G-1128).

```
cols = ["outlook", "temperature", "humidity", "wind"]
class_counts = df["play"].value_counts()          # Yes 9, No 5
prior = class_counts / len(df)                    # 9/14, 5/14
lookup = {col: pd.crosstab(df[col], df["play"]) / class_counts
          for col in cols}
```

`pd.crosstab` (G-1470) builds a **crosstab** (G-511): a table that counts how often each value appears with each class. Dividing each class column by the class size turns the counts into probabilities.

Figure 2 runs these two steps for the feature outlook:

1. **Count.** For each value and each class, count the matching days. Rain with "no" matches days 6 and 14, so the count is 2. Sunny appears on 3 "no" days and 2 "yes" days.
2. **Divide.** Divide the "no" column by 5 (the number of "no" days) and the "yes" column by 9. The counts become probabilities, for example:

$$P(\text{rain} \mid \text{no}) = 2/5$$

$$P(\text{sunny} \mid \text{yes}) = 2/9$$

and so on.



Figure 2: Training on one feature. The 14 days are counted into a crosstab, one cell at a time (the matching days are highlighted). Each column is then divided by its class size, 5 or 9, to give P(outlook | play)

Figure 3 shows the result of the same two steps for all four features, plus the priors: the full lookup table.

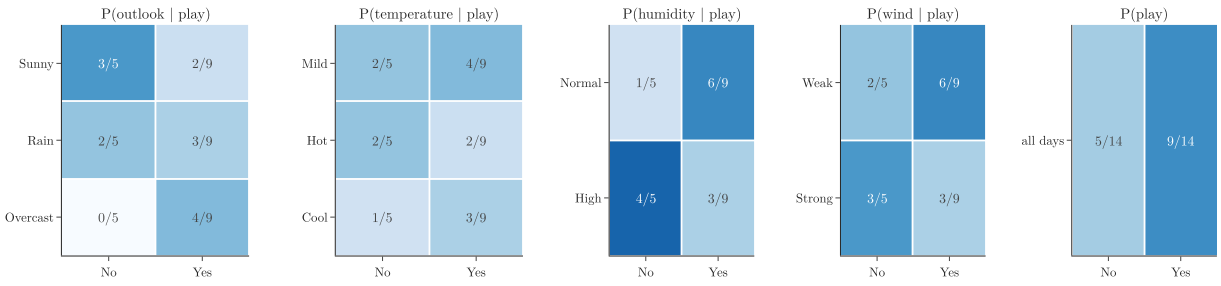


Figure 3: The full lookup table: P(value | play) for every feature, and P(play)

For example, of the 5 days without tennis, 4 had high humidity:

$$P(\text{high} \mid \text{no}) = 4/5$$

Of the 9 days with tennis, 6 had normal humidity:

$$P(\text{normal} \mid \text{yes}) = 6/9$$

5. Testing: look up and multiply

Key point: Sunny, hot, high, weak: yes scores 0.0071, no scores 0.0274. Predict no.

Python: Prediction.

```
def predict(day):
    scores = {}
    for c in ["Yes", "No"]:
        s = prior[c]
        for col, value in day.items():
            s *= lookup[col].loc[value, c]
        scores[c] = s
    return scores

predict({"outlook": "Sunny", "temperature": "Hot",
         "humidity": "High", "wind": "Weak"})
```

For the sunny, hot, high, weak day:

$$\text{yes: } \frac{9}{14} \times \frac{2}{9} \times \frac{2}{9} \times \frac{3}{9} \times \frac{6}{9}$$

$$\text{yes: } 0.0071$$

$$\text{no: } \frac{5}{14} \times \frac{3}{5} \times \frac{2}{5} \times \frac{4}{5} \times \frac{2}{5}$$

$$\text{no: } 0.0274$$

The "no" score is larger: **no tennis**. As probabilities (with more decimals, so the rounding does not show):

$$\frac{0.027429}{0.007055 + 0.027429}$$

$$= \frac{0.027429}{0.034484} = 0.795$$

So 79.5% no. Sunny weather and high humidity, both much more common on "no" days, decide it. This exact day is day 1 of the data, where tennis was not played, so the model was trained on it: the right answer here shows how the method works, not how well it predicts new days.

Figure 4 multiplies the factors in one at a time and shows the yes/no share after each. Watch "yes" start ahead at 64.3%, fall behind at "sunny", and end at 20.5%.

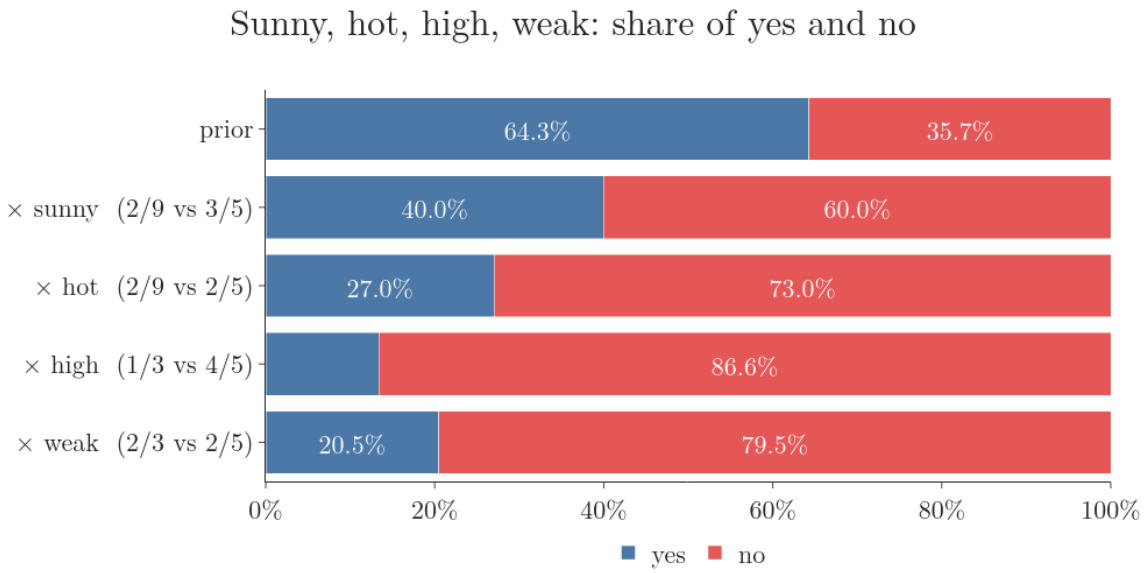


Figure 4: The sunny, hot, high, weak day, one factor at a time. Each bar is the yes/no share of the running product; brackets give P(value | yes) vs P(value | no)

6. The zero-frequency problem

> **Key point:** Overcast never occurred on a "no" day, so P(overcast | no) = 0, and any overcast day gets a "no" score of exactly 0.

One zero count met this problem on word counts. The same problem appears in the tennis data. Take an overcast, cool, normal-humidity day with weak wind, a combination that is not among the 14 days. In the data, it was overcast on 4 days, and tennis was played on all of them. So the chance of overcast on a "no" day is 0:

$$P(\text{overcast} \mid \text{no}) = 0/5 = 0$$

The whole "no" product is then 0:

$$\text{no: } \frac{5}{14} \times 0 \times \cdots = 0$$

Figure 5 runs the same build-up for this day. Watch the red "no" share vanish at "overcast" and never come back, whatever the later factors say.

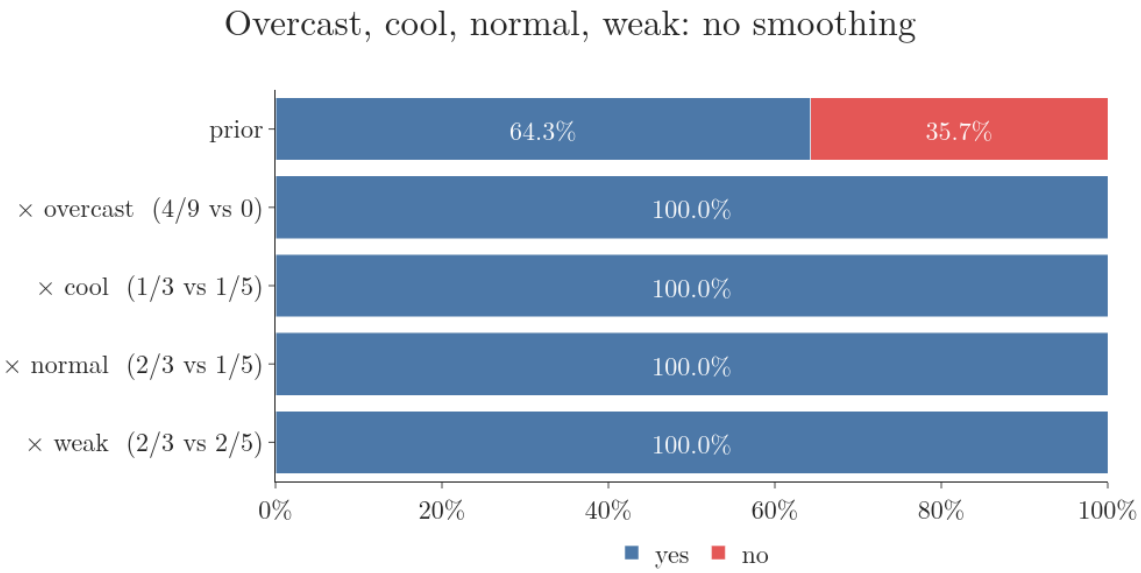


Figure 5: The overcast, cool, normal, weak day without smoothing. The factor P(overcast | no) = 0 sets the no share to 0 for good

The model is 100% sure tennis will be played, purely because one value never happened to appear with "no" in 14 observations. A single zero wipes out all the other evidence, like one veto in a committee that overrules every other vote. With real data and many features, unseen combinations of one value and one class are common, so this is a real problem: training data is never large enough to show every rare event (Manning et al. §13.2).

7. The fix: Laplace smoothing

> **Key point:** Add 1 to every count before dividing. No probability is ever exactly 0, and common values are barely affected.

Laplace smoothing (G-1045) (or **add-one smoothing**) adds a small count, usually 1, to every cell of each **crosstab** (G-511) before turning it into probabilities (Manning et al. §13.2):

$$P(\text{value} \mid \text{class}) = \frac{n_{\text{value}} + 1}{n_{\text{class}} + k}$$

Here n_{value} is the count of the value in the class, n_{class} is the class count and k is the number of values.

The added count is usually written α; here $\alpha = 1$. Step by step, for outlook given "no":

1. The counts are overcast 0, rain 2, sunny 3, out of 5 "no" days.
2. Add 1 to each: 1, 3, 4. The total grows by the number of values, 3, to 8.
3. Divide. Overcast becomes 0.125 instead of 0:

$$(0 + 1)/(5 + 3) = 0.125$$

Sunny becomes 0.5 instead of 0.6:

$$(3 + 1)/8 = 0.5$$

The denominator grows by k because each of the k values got one extra count. Dividing by the new total keeps the three probabilities adding up to 1:

$$\frac{1}{8} + \frac{3}{8} + \frac{4}{8} = \frac{8}{8} = 1$$

The priors are not smoothed: adding counts to feature values does not change how many days belong to each class.

Day	Without smoothing	With Laplace smoothing
sunny, hot, high, weak	79.5% no	68.8% no
overcast, cool, normal, weak	100% yes	96.4% yes

The predictions stay the same, but the model is no longer absolutely certain about anything it has seen only a few times.

Figure 6 puts the overcast day before and after smoothing side by side. Watch the "no" share: without smoothing it drops to 0 at "overcast"; with smoothing it shrinks to 14.3% but survives, ending at 3.6%.

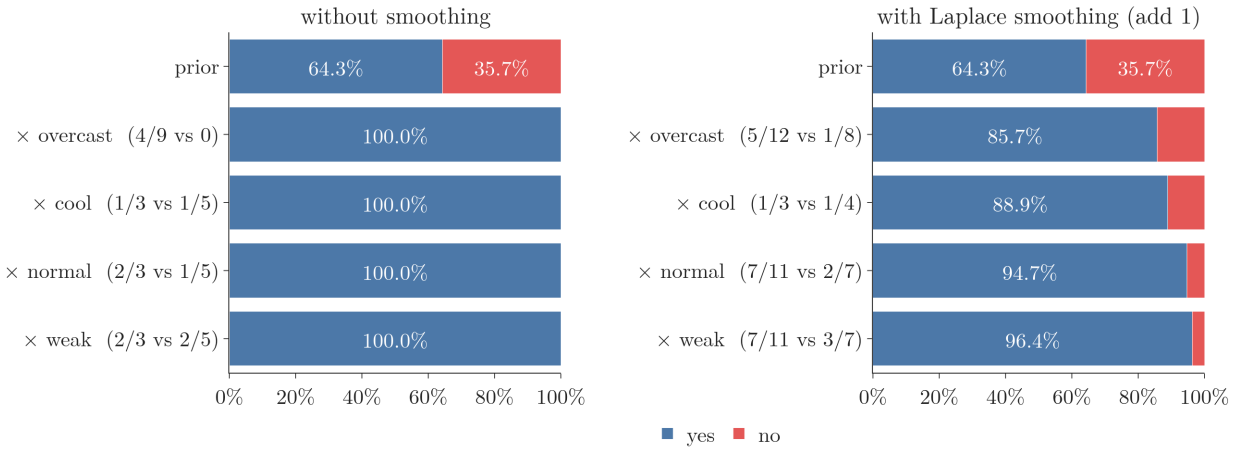


Figure 6: The overcast day without (left) and with (right) Laplace smoothing. Smoothed factors such as 1/8 for P(overcast | no) keep every class alive: 96.4% yes instead of 100%

Python: scikit-learn's CategoricalNB.

```
from sklearn.naive_bayes import CategoricalNB
from sklearn.preprocessing import OrdinalEncoder
```

```
enc = OrdinalEncoder()
X = enc.fit_transform(df[cols])        # categories -> 0, 1, 2
nb = CategoricalNB(alpha=1.0).fit(X, df["play"])
nb.predict_proba(enc.transform(new_days))
```

`CategoricalNB` is Naive Bayes for categorical features. It expects each feature's categories as the numbers 0, 1, ..., n − 1, so `OrdinalEncoder` first turns the words into those numbers (scikit-learn docs, `CategoricalNB`). Its `alpha` is the smoothing count, 1 by default (scikit-learn docs, `CategoricalNB`); with a tiny `alpha` it reproduces the unsmoothed numbers above exactly.

8. Summary

- Training: count with `pd.crosstab`, divide by the class sizes, store the **lookup table** (G-1128) (22 numbers here), so the data is read only once and every probability the formula could need is ready.
- Testing: multiply the prior by one looked-up probability per feature; the largest score wins, so prediction needs no recomputation.
- Sunny, hot, high, weak gives no (0.0274 against 0.0071), because sunny weather and high humidity are much more common on "no" days.
- A value never seen with a class gives a zero that overrides everything, because the score is a product; **Laplace smoothing** (G-1045) (add 1 to every count) prevents it, so the model is no longer 100% sure about something it saw only a few times. `CategoricalNB(alpha=1)` does this by default.
- So in code, Naive Bayes is two short steps, count then look up and multiply, plus smoothing to keep one unseen value from deciding the answer.

9. Sources

Built from

- CampusX, "Naive Bayes Classifier | Part 8 | Simple Example Code", YouTube, https://www.youtube.com/watch?v=DeeWsqoY4Eo
- StatQuest with Josh Starmer, "Naive Bayes, Clearly Explained!!!", YouTube, https://www.youtube.com/watch?v=O2L2Uv9pdDA (the zero count and adding $\alpha = 1$ to every count, sections 6 and 7)

Other references

- **Manning et al.:** Manning, C. D., Raghavan, P. and Schütze, H. *Introduction to Information Retrieval*. Cambridge University Press, 2008. Section 13.2, "Naive Bayes text classification".
- **scikit-learn docs:** `sklearn.naive_bayes.CategoricalNB` (alpha, input encoding), scikit-learn 1.9.

10. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Lookup table (G-1128)	The stored probabilities (class priors, and each feature value's probability within each class) that Naive Bayes computes once in training and then looks up to score new rows.
Zero-frequency problem (G-2149)	A probability of 0 for a value never seen with a class, which forces that class's score to 0.

Term	Meaning
Laplace smoothing (G-1045)	Adding a small count (usually 1) to every count so that no probability is 0.
pd.crosstab (G-1470)	pandas function that counts how often each pair of values from two columns occurs.
CategoricalNB (G-353)	scikit-learn's Naive Bayes for categorical inputs.
Prior (G-1565)	The probability of an event before any evidence is seen.
Likelihood (G-1086)	How probable the observed data is under given parameter values; read as a function of the parameters with the data fixed.
MAP rule (G-1157)	Predict the class that is most probable for the row, given its features (the class with the largest posterior probability; maximum a posteriori).
Crosstab (G-511)	A table that counts the rows for every pair of categories of two categorical columns, so we can see whether the two are related; pandas builds it with `pd.crosstab`. Also called a contingency table.